

Ops Control Tower

One person owns every problem. Four agents do the legwork.
Not a faster chatbot — the structure around the tickets.

detected



escalated



owner_acked



fix_confirmed



customers_notified



closed — earned, not declared

Q1 • WHAT PROBLEM

Ops runs the old way: one ticket at a time.

Ops integrates more information than any other role — yet it still works in specialist columns and handoffs. In the dataset, **38 of 100 tickets aren't support work at all**; they're cross-ticket problems wearing ticket costumes, and nothing reclaims them:

▲ BURST INCIDENT

Card outage, 4 days

14 tickets under 14 different subject lines — never declared as one incident.

▲ BURST INCIDENT

\$342k wires stuck

9 tickets, 8 customers; payroll and suppliers blocked; 5 of 9 unresolved.

▲ SLOW-BURN TREND

FX churn signal

10 customers over 3 weeks, CSAT 1.67 — invisible to any daily review.

◆ HIDDEN RISK

"Quick question"

An unrecognized admin — a likely account takeover, still open.

Ops doesn't need a faster version of the old way — **it needs an AI-native one.**

Q2 • WHY — REORGANIZE AROUND LOOPS, NOT FUNCTIONS

A working sample of the **transposed organization**.

BEFORE — DIVIDED BY FUNCTION

Specialists own columns; every problem travels by handoff. What the dataset shows:

	SUPPORT	PAYMENTS	ENGINEERING	PRODUCT
Card outage	rep ×14	rep	never told	
\$118k wire	rep ×2	rep	never told	
FX churn	rep ×11			never reached

One outage lived as **14 separate tickets for 3 days**. The churn signal was answered 11 times — and **never reached the team that owned it**.

AFTER — DIVIDED BY LOOP

One owner per loop, end to end; agents fill the non-native cells:

	SUPPORT	PAYMENTS	ENGINEERING	PRODUCT
Incident loop	sophie	Pattern Cmdr AI	Liaison → Alex AI	
Wire loop	sophie	Case Worker AI	Liaison AI	
Churn loop	sophie	Triage AI		→ Mei ✓

☒ human judgment (native) ☐ agent-executed

Same person from problem to closure — **the loop closes itself**. And this time, **the FX signal reaches product**.

WHY START IN OPS:

highest task volume

densest information surface

every ticket = a training signal

Q3 · HOW — ONE SHARED TEAM OF AGENTS

The team shares agents — nobody prompts alone.

Helpdesk seam

mock API replays the queue as a **real-time stream**; labels written back. CSV fallback.



Deterministic pipeline

LLM enrichment → validated schema
burst + trend detectors + precedence
explainable score — formula shown



War room (OpenMausBot, MIT)

Triage · Pattern Commander · Case Worker · Liaison + simulated colleagues from eng/product/compliance. One workspace = maximal shared context. The incident manager holds every gate.



Outputs that travel

incident cards · work packages
briefs → eng/product · OPS-xxx
filed
weekly brief + console
snapshot

🔄 **feedback loops: corrections · coaching · knowledge · skill proposals** → back into the pipeline (few-shot, overrides, merge rules, eval cases)

HUMAN GATES:

escalation — @Liaison go ahead

customer drafts — draft_approved

skill changes — approve SCP-xxx

audit every action → out/actions.jsonl · **--history TKT-2072** answers "what happened to this ticket?" in one command

Q3 · HOW — DETECTION IS LIVE, CLOSURE IS EARNED

Caught on the **third ticket** — then tracked to closed.

Aug 4 · 08:30	first decline report lands (TKT-2059)
Aug 4 · 15:43	INC-001 declared on the 3rd ticket — 7h13m in. The real team worked these as separate cases for 3 days
Aug 3 · 12:00	TRD-001 fires the moment the FX theme crosses the 14-day span — a pattern no daily review can see
Aug 10 · 11:00	"Quick question" arrives → RISK_PINNED instantly (possible account takeover)

INC-001 · card declines — walked to notified

detected

escalated

owner_acked

fix_confirmed

customers_notified · 410h

INC-002 · stuck wires — refuses to close

detected

escalated

cannot close — theme still active

Found is not fixed — the brief reports **time-to-close** beside **time-to-declare**.

Q3 · HOW — THE TEAM LEARNS, WITH AUTHORITY CHECKED

Four channels. Each with the right authority.

1 · CORRECTIONS

Ops-role-gated, enforced in code.
One correction = override + few-shot + eval case.

2 · COACHING

"From now on..." → one rule in NOTES.md. Capped at 20, git-audited, human-pruned.

3 · SELF-LEARNING

Nightly journals + weekly digests into KNOWLEDGE.md — facts only, never rules. Agents commit under their own git author.

4 · SKILL EVOLUTION

Agents propose frozen diffs; sophie approves in chat; the executor can apply only the recorded diff — humans-only by construction.

75 → 96%

segment accuracy · 2
corrections · leave-one-out

88 → 83%

same run caught a coaching
side effect

14/14 · 9/9 · 11/11

detection, zero false
positives

71

tests green · 24-ticket golden
eval

Q4 · KEY ASSUMPTIONS

The core assumption is a **team design**.

4 AGENT ROLES – EXECUTION

Triage

queue, routing, corrections

Pattern Commander

incident & trend cards

Case Worker

playbook work packages

Liaison

cross-team briefs, tracker

3 HUMAN ROLES – JUDGMENT (ONE OPERATOR WEARS ALL THREE HATS IN THIS DEMO)

Loop owner

runs the queue, makes the calls

Platform specialist

encodes expertise into playbooks & skills

Loop manager

reviews corrections, approves outbound

THE SEAMS WHERE THEY SPLIT AT SCALE:

correct_labels

approve_drafts

approve_escalations

approve_skill_changes

Also assumed: ticket text is the only system of record · monthly revenue proxies impact · synthetic data → no privacy concern in LLM calls

Q5 · WITH MORE TIME

DASHBOARD

The console snapshot becomes a continuously-updated hosted view: throughput, accuracy, backlog, live.

SKILL OPTIMIZATION

Systematic, driven by eval + correction history — the proposal channel already collects the raw material.

OUTBOUND REPLIES

Agents draft customer replies inside the helpdesk — gated by human approval before anything leaves the building.

PRODUCTION ADAPTER

The two-method seam exists; what remains is OAuth, webhooks, rate limits, idempotent write-backs.

Real agent leverage is structural, not conversational.

Try it in 2 seconds — `reports/console-snapshot.pdf` renders on GitHub · the HTML original opens locally · **30 minutes** — `scripts/setup_demo.sh` recreates the live war room (verified install, no sudo, offline by default)